

Callbacks and Composition

Document #: P1678R2
Date: 2019-10-06
Project: Programming Language C++
LEWG Library Evolution
Reply-to: Kirk Shoop
<kirkshoop@fb.com>

Contents

1	Changelog	2
2	Introduction	2
2.1	examples of callbacks	3
2.2	<code>async_accept</code> example	3
2.3	composing callbacks	3
3	Poll proposals	3
4	Motivation	4
4.1	Discover concepts by studying Data Structures and Algorithms	4
4.2	composition challenges	4
4.2.1	callback functions	4
4.2.2	functions taking callbacks	5
4.3	algorithm composition	5
4.3.1	a generic <code>retry()</code> algorithm	5
4.3.2	algorithm composition Examples	7
5	Function output	8
5.1	Values	8
5.2	Exceptions	8
5.3	Multiplexing	8
5.4	Contrast function-taking-a-callback with functions	9
6	Proposals	9
6.1	Default representation for new library callback	9
6.1.1	rational	9
6.2	Default representation for new library functions taking a callback	10
6.2.1	rational	11
7	Usage for the proposed representations	11
8	Composition for the proposed representations	12
9	Changes to the Standard	13
10	Representations	14
10.1	callback representations	14
10.1.1	value and error arguments style	14
10.1.2	completion handler style	15

10.1.3	<code>std::expected</code> style	17
10.1.4	multiple function style	17
10.2	function taking a callback representations	19
10.2.1	function argument style	19
10.2.2	return value style	20
10.2.3	Networking TS style	20
11	Credits	21
12	Appendices	21
12.1	<code>future_from</code> implementation	21
12.1.1	support <code>std::promise</code> as a receiver	22
12.1.2	implement <code>future_from</code>	22
12.2	<code>use_future</code> completion handler implementation	23
12.2.1	<code>use_future.hpp</code>	24
12.2.2	<code>impl/use_future.hpp</code>	27
12.3	[P1386R2] audio	44
12.3.1	events	44
12.3.2	buffers	47
13	References	49

1 Changelog

R2

- ☒ focus on LEWG
- ☒ add poll proposals
- ☒ reorganize paper

R1

- ☒ Use bibliography for Cologne paper references
- ☒ Example tony tables for `use_pipe`, `use_sender`
- ☒ Partial Success example
- ☒ add stack-frame analogy for callbacks
- ☒ use ‘handler’ instead of ‘token’
- ☒ code for `use_future`
- ☒ code for `future_from()`
- ☒ code for `retry()`
- ☒ add audio examples

2 Introduction

A goal of this paper is to select one callback pattern that can be used by default for functions and callbacks being added to C++ libraries. There will still be Invocables added that should not conform to this pattern. For example, The whole purpose of algorithms is to apply Invocable projections and Invocable predicates to data.

A non-goal of this paper is to explore or select the callback pattern for callback sequences. This is a goal of a later paper and this paper has been written with callback sequences in mind. callback sequences will be the basis of AsyncRange proposals, work well for streaming data over a network and also will apply to audio and ui events.

NOTE: callbacks are just as likely to be synchronous as asynchronous. While this paper will use an async function to explore the different styles for functions and callbacks - the motivations and proposals in this paper apply to both synchronous and asynchronous functions taking callbacks.

2.1 examples of callbacks

callbacks are common in the standard library. `std::visit` and `std::for_each` are algorithms that take a callback. The `std::thread` constructor takes a callback. `std::promise` is a callback, and the proposed `std::experimental::future::then()` takes a callback. callbacks are fundamental to the Networking TS and the Executors proposal. C++20 coroutines generate callbacks and a state-machine for calling them so that users can write code without explicit callbacks.

The callback pattern is so common because a function call is a great way to transfer data (eg. the function passed to `std::visit`), change execution context (eg. the function passed to `std::thread`) and modify the behaviour of an algorithm (eg. the predicate passed to `std::sort`).

2.2 async_accept example

Examples of callbacks used for async functions can be found in the Networking TS [N4771]. For the purpose of comparison, one async function and its completion signature will be used to explore the different designs presented in this paper. The signature for `async_accept()`, as defined in [N4771] is:

```
template<
    typename SocketService,
    typename AcceptHandler>
void-or-deduced async_accept(
    basic_socket< protocol_type, SocketService > & peer,
    endpoint_type & peer_endpoint,
    AcceptHandler handler);
```

This signature indicates that there is a single function where the last argument is used for the callback that will be provided the result and the preceding arguments contain the inputs needed to generate the result.

The completion signature for `async_accept()` is `void(error_code ec, socket_type s)`. This completion signature indicates that there is a single function where the first argument is used for the error and the second argument is used for the result that will be called for errors and results.

2.3 composing callbacks

All of the functions and callbacks mentioned have different signatures. The primary effect of all these disparate signatures is that composition of one function and callback to another, say `std::promise` and `void(error_code ec, socket_type s)`, must be written by hand. For all the functions and callbacks that already exist, this manual adaptation cannot be avoided. Proposing a solution for new callbacks that is composable, is the motivation for this paper.

3 Poll proposals

- [] Which of these viewpoints is preferred by LEWG?
 - There is no common surface for networking, file, audio, execution, and algorithms and we derive each surface from the needs of each area

- There is a common surface for networking, file, audio, and execution and we derive that common surface from what the algorithms need, not from what the cpu execution, audio, file or networking libraries need
- [] Going forward, concepts and types for [callback representations](#) in the C++ library will model the [multiple function style](#)
- [] Going forward, concepts and types for [function taking a callback representations](#) in the C++ library will model the [return value style](#)

4 Motivation

The STL ended a long period where each implementation of a list or dynamic array or string had a unique surface. The different representations of containers and element traversal shared enough terminology to cause confusion and shared enough semantics to be dangerous when composed with algorithms and other container implementations.

Functions taking callbacks, at the moment, are exactly where containers were before the STL. Just like the STL needed to define stable Container & Range concepts to be used to compose different containers and algorithms together, C++ needs to define stable Sender & Callback concepts to be used to compose different tasks and executors and algorithms together.

4.1 Discover concepts by studying Data Structures and Algorithms

Stepanov, A., 2006, *Notes On Programming, Lecture 13. Iterators* ([pdf](#))

“It might be surprising to you, but I find the subject of iterators extremely hard to teach. The main reason is that I find the notion self-evident and all the fundamental design decision non-negotiable. But I also know that somehow even those people who are very enthusiastic about iterators, the STL “specialists,” often demonstrate that their understanding of iterator fundamentals is quite shaky.

The fact that I find the concept so self-evident is the result of many years of trying alternatives and finding that they do not work.

In some sense the only way for someone to fully understand why they have to be the way they are is by trying hundreds of different algorithms and finding the abstraction that allows the most beautiful and efficient representation of them.

As a matter of fact, the only way of finding a useful abstraction is by trying to write code in terms of it. Sadly enough, people tend to define abstractions faster than they try them. There is even a pernicious idea of having “architects,” which are often people who produce abstractions without writing code. It is a worthwhile thing to remember that the most successful abstraction ever introduced in computer science – an abstraction of a file as a sequence of bytes with the help of which Ken Thompson revolutionized systems design – did not originate as an abstraction at all, but as a specific data structure for implementing files.

Good abstractions come from efficient algorithms and data structures and not from “architectural” considerations."

4.2 composition challenges

4.2.1 callback functions

Callbacks are challenging for composition because they all have a different shape. Signatures such as:

- completion/termination - `void()`
- error and values - `void(auto ec, auto... v)`

- errors - `void(auto ec)`
- values - `void(auto... v)`

All these callback patterns exist and make composition of callbacks a bespoke, repetitive, and error-prone task.

A callback that takes error and value arguments, must support invalid or empty states for each argument, because the same function will be called for error and success. When error and result are delivered to the same function all implementations of these callbacks are required to check the arguments for validity before using the arguments. These checks introduce branches, which can be particularly expensive instructions.

Another way to represent an empty state is to use `std::optional` explicitly on all the args so that the value types used as callback arguments are not required to support an invalid or empty state, but the branches remain the same and the codegen for `std::optional` is added.

NOTE: `std::error_code` supports an ‘empty’ state. The empty state for a `std::error_code` is the success code.

4.2.2 functions taking callbacks

A friction point for functions that take callbacks, is that the callback is placed in different positions in the argument list of the function:

- first - `std::visit(callback, variant...)`
- last - `std::for_each(begin(r), end(r), callback)`

These different callback argument patterns exist and make composition of functions taking callbacks a bespoke, repetitive and error-prone task.

A limitation of functions, that take callbacks as the last argument, is that this affects valid signatures for and overloading of the functions. Having a fixed last argument, requires adding overloads when defaulting the values of the non-callback arguments. Having a fixed last argument, requires that the overloads be constrainable so that the callback can be reliably distinguished from the non-callback arguments. Having a fixed last argument, prevents using variadic non-callback arguments, which is why `std::visit` places the callback as the first argument.

4.3 algorithm composition

Composition is improved when callbacks have a regular shape. A regular shape for callbacks and functions taking callbacks allows composition to be generically implemented in algorithms.

If all callbacks had a regular shape then it would allow algorithms like `std::this_thread::sync_get`, `std::when_all`, `std::when_any`, `std::retry`, `std::repeat`, `std::take_until`, `std::timeout`, `std::at`, `std::sequenced`, `std::on`, `std::via`, `std::just`, `std::defer`, `std::transform`, etc.. to compose different functions taking callbacks.

4.3.1 a generic `retry()` algorithm

When Callbacks have a regular shape, generic algorithms can synthesize new Callbacks to chain new semantics onto arbitrary producers.

NOTE: For the purpose of comparison this paper will use the Callback naming specified in [P1660R0] as an example of [multiple function style](#). The names chosen for a particular expression of the [multiple function style](#) do not affect this proposal.

Table 1: generic `retry()` algorithm example (simplified for clarity)

function	pipe operator
<pre> namespace retry_alg { template<class S, class C> struct retry_callback { S s_; C c_; void operator()(auto... vn) { c_(vn...); } void error(auto) noexcept { s_.submit(*this); } void done() noexcept { c_.done(); } }; template<class S> struct retry_sender { S s_; void submit(Callback auto c) { s_.submit(retry_callback<S, decltype(c)>{s_, c}); } }; // ----> // ----> // ----> // ----> // ----> struct fn { auto operator()(Sender auto s) { return retry_sender<decltype(s)>{s}; } }; constexpr inline retry_alg::fn retry{}; </pre>	<pre> namespace retry_alg { template<class S, class C> struct retry_callback { S s_; C c_; void operator()(auto... vn) { c_(vn...); } void error(auto) noexcept { s_.submit(*this); } void done() noexcept { c_.done(); } }; template<class S> struct retry_sender { S s_; void submit(Callback auto c) { s_.submit(retry_callback<S, decltype(c)>{s_, c}); } }; struct pipe_fn { auto operator()(Sender auto s) { return retry_sender<decltype(s)>{s}; } }; struct fn { auto operator>() { return pipe_fn{}; } }; constexpr inline retry_alg::fn retry{}; </pre>

4.3.1.1 differences between function and pipe operator composition

The **function** composition example produces a `retry()` function that must be passed the input sender that will be retried until it succeeds or is cancelled.

The **pipe operator** composition example produces a `retry()` function that takes no arguments and returns a function that must be passed the input sender that will be retried until it succeeds or is cancelled.

This is the only difference between these two composition models.

4.3.1.2 Notes

The retry algorithm takes an input sender to repeatedly submit, and an output callback that wants the result of the input sender, once it succeeds or is cancelled.

A retry sender is synthesized once the input sender is provided. When `submit()` is invoked on the retry sender, thus providing the output callback, a retry callback is synthesized.

The retry callback is passed to `submit()` on the input sender and passes everything except errors to the output callback. When `error()` is invoked on the retry callback, a copy of the retry callback is passed to `submit()` on the input sender.

This continues until the input sender succeeds or is cancelled.

4.3.2 algorithm composition Examples

Composition of algorithms that synthesize callbacks might result in code that looked like this:

Table 2: using `at`, `timeout` and `when_any` to get fresh data and update a cache or fallback to the cache if the data takes too long

function	pipe operator
<pre> auto get_data() { auto fallback = sequenced(at(ex, now() + 4s), cached_request(ex)); return timeout(when_any(update_cache(network_request(ex)), move(fallback)), at(ex, now() + 5s)); } </pre>	<pre> auto get_data() { auto fallback = at(ex, now() + 4s) sequenced(cached_request(ex)); return when_any(network_request(ex) update_cache(), move(fallback)) timeout(at(ex, now() + 5s)); } </pre>

Table 3: using `when_all` to compose async (`get_data`) and sync (`std::visit`) callbacks with `retry` and `take_until`

function	pipe operator
<pre> auto foo(stop_token s) { return take_until(when_all(retry(get_data()), visit(v_)), s); } </pre>	<pre> auto foo(stop_token s) { return when_all(get_data() retry(), visit(v_)) take_until(s); } </pre>

Table 4: using `defer` and `repeat` to keep a connection alive until cancelled

function	pipe operator
<pre> auto keep_alive(stop_token s) { return take_until(repeat(sequenced(defer([ex]() { return at(ex, now() + 5s); }), ping_request(ex))) s); } </pre>	<pre> auto keep_alive(stop_token s) { return defer([ex]() { return at(ex, now() + 5s); }) sequenced(ping_request(ex)) repeat() take_until(s); } </pre>

5 Function output

Here is a short description of the options currently in the language for functions to return values. These options boil down to three channels; return value, out-parameter arguments, and throwing exceptions.

5.1 Values

In C, there are three ways to communicate a result:

- return a value
- set value(s) into out-parameter(s)
- call a parameter, that is a function, with arguments(s)

5.2 Exceptions

C++ added a third mechanism for communicating a result - throwing exceptions. Adding exception throwing as a separate communication channel allowed code to focus on the path of success and delegate the responsibility for exception handling to the caller by default. C++ made support for exceptions implicit. Functions do not have a mechanism to opt-in to exception support. Functions can opt out of emitting exceptions using `noexcept`, but the compiler still is responsible for ensuring that an attempt to throw an exception in a `noexcept` function will result in a call to `std::terminate`.

5.3 Multiplexing

These mechanisms can be multiplexed and de-multiplexed, with additional overhead in code size and runtime.

Examples of mux for return values and out-parameters:

- `optional<T>` allows return without a result.
- `expected<E, T>` allows an error to be returned without an exception.
- `expected<E, optional<T>>` allows an error to be returned without an exception and for nothing to be returned.
- `expected<optional<variant<tuple<Tn0...>, tuple<Tn1...>, ...>>, E>` allows the parameters that are supported by one of an overload set of callback functions to be returned as a value and an error to be returned without an exception and for nothing to be returned.

Potential syntax to simplify the code that needs to be written to demux these values can be found in the proposal for pattern matching [P1371R0].

NOTE: while `expected`, `variant` and `tuple` all have corresponding C++ language features (exception & return value have `expected`, overload set of functions have `variant`, and multiple arguments to a function have `tuple`), `optional` does not have a language representation. `Pointer` is not a language representation `asoptional` is a super-set of `Pointer`, because `optional` stores the value when it is valid, while `Pointer` does not.

5.4 Contrast function-taking-a-callback with functions

- A function-taking-a-callback is invoked from a stack frame that may not exist when return-value|exception is emitted
- The only remaining fragment of the stack frame that invoked the function-taking-a-callback is the callback argument
- The signals return-value|exception that would be delivered to the stack frame that invoked a function-taking-a-callback must be delivered to the callback argument

6 Proposals

The goal of this paper is to provide the rational for selecting one concept for functions taking callbacks, and one concept for callbacks.

Detailed descriptions of the alternatives can be found in the [representations](#) section.

6.1 Default representation for new library callback

Of the [callback representations](#), the [multiple function style](#) is the one proposed by this paper, as the style to be adopted as a default style for all new callbacks in libraries for C++. The [multiple function style](#) was chosen even though the [completion handler style](#) is already established as the callback mechanism in the Networking TS. The rational for this proposal follows. For the purpose of comparison this paper will use the Callback naming specified in [P1660R0] as an example of [multiple function style](#). The names chosen for a particular expression of the [multiple function style](#) do not affect this proposal.

Table 5: compare Invocable callback usage for the existing [completion handler style](#) with the proposed [multiple function style](#)

Existing	Proposed
<pre>auto async_accept_completion = [](error_code ec, socket_type s){};</pre>	<pre>auto async_accept_completion = callback([](error_code ec){}, [](socket_type s){});</pre>

6.1.1 rational

The data behind the rational is spread throughout the [callback representations](#) section. To make the comparison easier the main points are represented in tables here:

Table 6: compare tradeoffs selected by [completion handler style](#) with [multiple function style](#)

	completion handler style	multiple function style
1.	the callback is an Invocable or a completion handler that provides an Invocable that will convert to some other style of callback	a callback is an object that is an Invocable and allows calls to error() and done() functions. Converting to other callback styles is a separate concern
2.	single functions will tend to be implemented as objects that have additional features like allocators and executors, though this implementation approach is not required. simple functions are allowed	callback objects will tend to be implemented by defining methods on an object, though this implementation approach is not required. Niebloid's will be used to allow free-function customizations for arbitrary types
3.	calls to the single function callback may have different execution guarantees that will depend on the runtime value of the parameters. One example is when an error may not be deliverable from the same execution context that is used to deliver the result	calls to each function on a callback may have different execution guarantees specified by the caller.
4.	argument types are required to represent invalid or empty states	argument types are not required to represent invalid or empty states
5.	the single function callback is required to add branches and checks for errors and validity before accessing the arguments	the functions are not required to add branches and checks for errors or validity
6.	all types are passed as function arguments with no required packing/unpacking	all types are passed as function arguments with no required packing/unpacking
7.	a completion handler does not support overloads , something like <code>std::variant</code> would be required to support multiple result types. a single function does support overloads.	each function supports overloads that allow different types to be supported without use of <code>std::variant</code> . this allows both error and value to have independent overloads
8.	a completion handler does not support overloads , something like <code>std::optional</code> or <code>std::variant<std::tuple<>...></code> would be required to support multiple result types. a single function does support overloads.	each function supports overloads that allow different numbers of arguments to be supported without use of <code>std::optional</code> or <code>std::variant<std::tuple<>...></code>

6.2 Default representation for new library functions taking a callback

Of the [function taking a callback representations](#), the [return value style](#) is the one proposed by this paper, as the style to be adopted as a default style for all new functions taking callbacks in libraries for C++. The [return value style](#) was chosen even though the [Networking TS style](#) is already established. The rational for this proposal follows. For the purpose of comparison this paper will use the Sender naming specified in [P1660R0] as an example of [return value style](#). The names chosen for a particular expression of the [return value style](#) do not affect this proposal.

Table 7: compare Invocable callback usage for the existing [Networking TS style](#) & [completion handler style](#) with the proposed [return value style](#) & [multiple function style](#)

Existing	Proposed
<pre> async_accept(socket, endpoint, [](error_code ec, socket_type s){}); </pre>	<pre> std::submit(async_accept(socket, endpoint), std::callback([](error_code ec){}, [](socket_type s){})); </pre>

6.2.1 rational

The data behind the rational is spread throughout the [function taking a callback representations](#) section. To make the comparison easier the main points are represented in tables here:

Table 8: compare tradeoffs selected by [Networking TS style](#) with [return value style](#)

	Networking TS style	return value style
1.	adding the callback as the last argument makes overloading the function more restrictive than plain functions	returning a value that can attach the callback is no more restrictive than plain functions
2.	adding the callback as the last argument prevents variadic arguments to the function	returning a value that can attach the callback does not prevent variadic arguments to the function
3.	adding the callback as the last argument makes it hard to distinguish callback and non-callback arguments	returning a value that can attach the callback clearly distinguishes callback and non-callback arguments
4.	a particular completion handler can implement <code>async_initiate</code> to defer using the last argument and the return value	a function can directly implement defer using the return value
5.	each completion handler implementation is allowed to alter the form of all functions that use it	functions returning a value that can attach the callback have a stable form that makes it easier to write generic code

7 Usage for the proposed representations

Table 9: compare coroutine usage for the existing [Networking TS style](#) & [completion handler style](#) with the proposed [return value style](#) & [multiple function style](#)

Existing	Proposed
<pre>socket_type t = co_await async_accept(socket, endpoint, useAwaitable);</pre>	<pre>socket_type t = co_await async_accept(socket, endpoint); // <---</pre>

Table 10: compare future & get usage for the existing [Networking TS style](#) & [completion handler style](#) with the proposed [return value style](#) & [multiple function style](#)

Existing	Proposed
<pre>socket_type t = async_accept(socket, endpoint, useFuture).get();</pre>	<pre>socket_type t = std::future_from(async_accept(socket, endpoint)).get();</pre>

NOTE: The [future_from implementation](#) (~2 pages) and the [use_future completion handler implementation](#) (~20 pages) can be found in the [Appendices](#).

Table 11: compare Invocable callback usage for the existing [Networking TS style](#) & [completion handler style](#) with the proposed [return value style](#) & [multiple function style](#)

Existing	Proposed
<pre>async_accept(socket, endpoint, [](error_code ec, socket_type s){});</pre>	<pre>std::submit(async_accept(socket, endpoint), std::callback([](error_code ec){}, [](socket_type s){}));</pre>

8 Composition for the proposed representations

Table 12: compare function composition for the existing [Networking TS style](#) & [completion handler style](#) with the proposed [return value style](#) & [multiple function style](#)

Existing	Proposed
<pre> auto get_data() { auto fallback = sequenced(use_sender, at(ex, now() + 4s, use_sender), cached_request(ex, use_sender)); return timeout(when_any(use_sender, update_cache(network_request(ex, use_sender), use_sender), move(fallback)), at(ex, now() + 5s, use_sender)); } </pre>	<pre> auto get_data() { auto fallback = sequenced(// <--- at(ex, now() + 4s), cached_request(ex)); return timeout(when_any(// <--- update_cache(network_request(ex)), // <--- move(fallback)), at(ex, now() + 5s)); } </pre>

Table 13: compare pipe composition for the existing [Networking TS style](#) & [completion handler style](#) with the proposed [return value style](#) & [multiple function style](#)

Existing	Proposed
<pre> auto get_data() { auto fallback = at(ex, now() + 4s, use_pipe) sequenced(use_pipe, cached_request(ex, use_pipe)); return when_any(use_pipe, network_request(ex, use_pipe) update_cache(use_pipe), move(fallback)) timeout(at(ex, now() + 5s, use_pipe), use_pipe); } </pre>	<pre> auto get_data() { auto fallback = at(ex, now() + 4s) sequenced(cached_request(ex)); // <--- // <--- return when_any(// <--- network_request(ex) update_cache(), move(fallback)) timeout(at(ex, now() + 5s)); // <--- // <--- } </pre>

9 Changes to the Standard

If this proposal is accepted then additional papers could add overloads to some existing functions that take callbacks. For example:

NOTE: There are cases, like the following, where usage of the new overloads is not as succinct as the current function definition. The motivation for adding these overloads would be to enable direct composition with algorithms and other functions taking callbacks.

Table 14: demonstrate potential new overload for `std::visit` that will make composition easier

Existing	New
<pre>std::variant<int, long, std::string> v{42}; std::visit(overloaded { [](auto arg) {}, [](double arg) {}, [](const std::string& arg) {}, }, v);</pre>	<pre>std::variant<int, long, std::string> v{42}; std::submit(std::visit(v), std::callback(overloaded { [](auto arg) {}, [](double arg) {}, [](const std::string& arg) {}, }));</pre>

Table 15: demonstrate potential new overload for `std::async` that will make composition easier

Existing	New
<pre>std::async([](int i, const std::string& str){}, 42, "Hello");</pre>	<pre>std::submit(std::async(42, "Hello"), std::callback([](int i, const std::string& str){}));</pre>

10 Representations

10.1 callback representations

There are infinite representations of callbacks. A few of these will be described in this paper and will be explored using `async_accept()` as defined in [N4771] and its completion signature `void(error_code ec, socket_type s)`.

10.1.1 value and error arguments style

Using separate arguments to a callback to represent error and value channels involves some unfortunate tradeoffs. The completion signature `void(error_code ec, socket_type s)` for `async_accept()` in [N4771] implies that the `socket_type` must support an invalid or empty state when `ec` contains an error. This style requires that all the parameters used in a completion signature support invalid or empty states, because the same function will be

called for error and success. This requires all implementations of callbacks to check the arguments for validity before using the arguments. These checks introduce branches, which can be particularly expensive instructions.

10.1.2 completion handler style

The Networking TS [N4771] uses the completion handlers described in [N4045] to overload the callback argument to async functions like `async_accept()`. If the callback argument is a function matching the completion signature (eg. `void(error_code ec, socket_type s)` for `async_accept()`), then the function is used as the callback (in other words, the completion handler style subsumes the [value and error arguments style](#)). On the other hand, if the argument is a completion handler, then the completion handler implements a callback that matches the completion signature for the async function and uses the implementation of that callback to convert from [value and error arguments style](#) to some other callback representation (eg. `std::promise type`).

Therefore, a completion handler is a callback adaptor factory or an Invocable callback. [N4045] describes some machinery to hide the differences while implementing an async function.

[N4045] 9.1.1 contains this example for implementing an async function using machinery to hide the differences:

```
template <class Buffers, class CompletionHandler>
auto async_foo(socket& s, Buffers b, CompletionHandler&& handler) {
    async_completion<CompletionHandler,
        void(error_code, size_t)> completion(handler);
    // ..
    return completion.result.get();
}
```

[N4045] 9.2 discusses the implementation of a block completion type that uses `std::future::get()` to block the caller until the result is available. The implementation of the completion handler is way too long to include here. To get a sense of the size of a completion handler, an implementation of the [use_future completion handler implementation](#), from the Networking TS [N4771], is in the [Appendices](#) of this paper.

Here is the usage for `async_accept` with the block completion handler:

```
socket_type t = async_accept(socket, endpoint, block);
```

With [N4045], the callback form would be something like:

```
async_accept(socket, endpoint, [](error_code ec, socket_type s){});
```

With [N4045], the future & get form would be something like:

```
socket_type t = async_accept(socket, endpoint, use_future).get();
```

Note: an implementation of the [use_future completion handler implementation](#), from the Networking TS [N4771], is in the [Appendices](#) of this paper.

With [N4045], the coroutine form would be something like:

```
socket_type t = co_await async_accept(socket, endpoint, use_awaitable);
```

Having `async_accept()` take so many forms does affect reading code and writing generic code. Sometimes `async_accept()` returns the result, sometimes an object, and sometimes void, readers will need to adjust to seeing the same function take on different forms. Generic code is more complicated - to wrap up an async function like `async_accept()` in a generic function, the generic function must pick a completion handler to use while customizing the call to the async function but the completion handler that the generic function picks might not compose well or efficiently with the completion handler passed to the generic function that must ultimately communicate the result to the caller.

Underneath each of the forms generated by an async function that uses a completion handler would be a function of the [value and error arguments style](#) that was implemented by the completion handler. As mentioned, the [value](#)

and error arguments style requires that all arguments have an invalid or empty state.

The completion handler appears to add some additional constraints on the completion signature. One constraint is that, if there is an error argument, it must be the first argument. Another constraint is that the error argument must be the `error_code` or `exception_ptr` type. Also, to represent partial success an api would have to define a completion signature similar to `void(error_code, error_code, T...)`.

These constraints seem to be necessary because completion handlers like `use_future` and `use_awaitable` need to be able to distinguish between the error argument and the result argument(s), and need to be able to depend on how to convert the error argument into a thrown exception. Library functions can be built to generalize these additional constraints, much as completion handlers themselves are used to overload the meaning of a callback argument to an async function.

The semantics for `use_future` are described in this table from [N4771]

Table 16: [N4771] Table 10 - `async_result<use_future_t, Result(Args...)>` semantics

N	U0	R::return_type	F::operator() effects
0		<code>future<void></code>	None.
1	<code>error_code</code>	<code>future<void></code>	If <code>a0</code> evaluates to true, atomically stores the exception pointer produced by <code>make_exception_ptr(system_error(a0))</code> in the shared state.
1	<code>exception_ptr</code>	<code>future<void></code>	If <code>a0</code> is non-null, atomically stores the exception pointer <code>a0</code> in the shared state.
1	all other types	<code>future<U0></code>	Atomically stores <code>forward<A0>(a0)</code> in the shared state.
2	<code>error_code</code>	<code>future<U1></code>	If <code>a0</code> evaluates to true, atomically stores the exception pointer produced by <code>make_exception_ptr(system_error(a0))</code> in the shared state; otherwise, atomically stores <code>forward(a1)</code> in the shared state.
2	<code>exception_ptr</code>	<code>future<U1></code>	If <code>a0</code> is non-null, atomically stores the exception pointer in the shared state; otherwise, atomically stores <code>forward<A1>(a1)</code> in the shared state.
2	all other types	<code>future<tuple<U0, U1>></code>	Atomically stores <code>forward_as_tuple(forward<A0>(a0), forward<A1>(a1))</code> in the shared state.
2	<code>error_code</code>	<code>future<tuple<U1, ... , UN-1>></code>	If <code>a0</code> evaluates to true, atomically stores the exception pointer produced by <code>make_exception_ptr(system_error(a0))</code> in the shared state; otherwise, atomically stores <code>forward_as_tuple(forward<A1>(a1), ... , forward<AN-1>(aN-1))</code> in the shared state.

N	U0	R::return_type	F::operator() effects
>2	<code>exception_ptr</code>	<code>future<tuple<U1, ... , UN-1>></code>	If <code>a0</code> is non-null, atomically stores the exception pointer in the shared state; otherwise, atomically stores <code>forward_as_tuple(forward<A1>(a1), ... , forward<AN-1>(aN-1))</code> in the shared state.
>2	all other types	<code>future<tuple<U0, ... , UN-1>></code>	Atomically stores <code>forward_as_tuple(forward<A0>(a0), ... , forward<AN-1>(aN-1))</code> in the shared state.

These semantics attempt to split the error and values arguments apart and then route them to the `set_value()` and `set_exception()` methods on a promise.

Obvious limitations:

- only `exception_ptr` & `error_code` are recognized as error types and only when one of them is the first argument.
- partial success requires an api to specify a completion signature with two error arguments, like: `void(error_code, error_code, T...)` because the first error argument will be forwarded to `set_exception()` in which case `set_value()` will not be called.

NOTE: The full implementation of these semantics can be found in the [Appendices](#) of this paper. see [use_future completion handler implementation](#)

10.1.3 `std::expected` style

Another callback pattern is to combine the value and error into one argument. The completion signature for the `async_accept()` example might change to look something like `void(expected<error_code, socket_type> e)`.

This style does not require `socket_type` to support an invalid or empty state because it does not need to be constructed when there is an error. The branches required by the [value and error arguments style](#) are still required in this style, because the same function will be called for error and success.

There is also an additional cost in the codegen for packing and unpacking `std::expected`. The cost for `std::expected` is not as bad as when the value is a `std::tuple` or a `std::variant` of `std::tuples`, but still worse than when it is an plain argument to the function. For instance, something that transforms the result from one type to another has to check the error, unpack the result or error and repack the transformed result or original error into the outgoing expected type.

10.1.4 multiple function style

Some of the tradeoffs encountered when mixing errors and results into the same ‘channel’ (where function arguments and function results are both channels for communication with a function), motivated the creation of the C++ exception channel. C++ exceptions do not require the implementation of a function to check for the validity of function return values before using them and do not require that function return values support invalid or empty states (basically re-implementing `std::optional` in each type) nor require the use of types that combine error/value alternatives like `std::expected`.

Using multiple functions for error and result is equivalent to the separation of `return value` and `throw/catch` in the language. Using multiple functions for error and result produces very different tradeoffs than when mixing error and result together in one function. The [std::promise type](#) is an example of using multiple functions for error and result that already exists.

10.1.4.1 `std::promise` type

The `std::promise` type provides the member functions `set_value(T)|set_value()` and `set_exception(std::exception_ptr)`

- `set_value(T)|set_value()` is only called when there is result. Thus `T` does not need to support an invalid or empty state and implementations of `set_value` are not required to check for errors and thus no branch is added for that check.
- `set_exception(std::exception_ptr)` is only called when there is an error. Thus implementations of `set_exception` are not required to check for success and thus no branch is added for that check.

10.1.4.2 concepts

A challenge with the `std::promise` type is that it is a type with only one implementation, whereas callbacks are intended to be a concept or signature with many implementations. There are several examples of concepts that use multiple functions for error and result. These concepts primarily differ only in the names of the concepts and the names of the functions.

- Reactive Extensions defines the Observer concept which has been implemented in many different languages including C++. The `rxcpp` implementation uses the names `Observer::on_next(T)`, `Observer::on_error(std::exception_ptr)` and `Observer::on_completed()`
- [P1055R0] defines the Single concept using the names `Single::value(T)`, `Single::error(E)` and `Single::done()`
- [P1341R0] defines the Receiver concept using the names `Receiver::value(Tn...)`, `Receiver::error(E)` and `Receiver::done()`. The `pushmi` library has an implementation of the Receiver concept.
- [P1660R0] defines the Callback concept that subsumes the Invocable and Fallback concepts resulting in the names `Invocable::operator()(Tn...)`, `Fallback::error(E)` and `Fallback::done()`. [P1660R0] includes an example implementation.

NOTE: see ([P1677R0], [latest](#)) which contains a justification, for the existence of, and some uses for, the `done()` method.

The Callback concept defined in [P1660R0] has been gaining support in SG1 recently. When error and success are distinct, a completion object for the `async_accept()` example might change to look something like:

```
struct async_accept_completion {
    void operator()(socket_type s) && noexcept;
    void error(error_code) && noexcept;
    void error(exception_ptr) && noexcept;
    void done() && noexcept;
};
```

Where:

- `operator()` is only called for success
- `error()` is only called for failure
- `done()` is only called for neither-a-result-nor-an-error (see [P1677R0], [latest](#))

When partial success is supported, a completion object for the `async_accept()` example might change to look something like:

```
struct async_accept_completion {
    void operator()(socket_type s) && noexcept;
    void operator()(error_code, socket_type s) && noexcept;
    void error(error_code) && noexcept;
    void error(exception_ptr) && noexcept;
    void done() && noexcept;
};
```

Note how the overloads of the call operator and the error method allow dedicated code paths for failure, success and partial success.

With [P1660R0] the Callback form would be something like:

```
std::submit(  
    async_accept(socket, endpoint),  
    std::callback(  
        [](socket_type s){},  
        [](error_code ec){})  
    );
```

With [P1660R0] the future & get form would be something like:

```
socket_type t = std::future_from(async_accept(socket, endpoint)).get();
```

With [P1660R0] the coroutine form would be something like:

```
socket_type t = co_await async_accept(socket, endpoint);
```

In all of these `async_accept` does not change form. `async_accept` always returns a type that matches a concept that other functions can adapt. The adaption is not a new mechanism that is injected into the implementation of `async_accept`. Adaption is just a function that is passed the result of `async_accept` and adapts it to some other callback representation. `std::submit`, `std::future_from`, and `operator co_await()` are all external to the `async_accept` function and each of those functions has a stable form across all usage.

10.2 function taking a callback representations

In practice there appear to be very few representations of functions taking callbacks. These few will be described in this paper and will be explored using `async_accept()` and its signature, as defined in [N4771]:

```
template<  
    typename SocketService,  
    typename AcceptHandler>  
void-or-deduced async_accept(  
    basic_socket< protocol_type, SocketService > & peer,  
    endpoint_type & peer_endpoint,  
    AcceptHandler handler);
```

10.2.1 function argument style

Using separate arguments to a function to represent callback and input channels involves some unfortunate tradeoffs. The signature for `async_accept()` defined in [N4771] as it might look if only an Invocable callback argument was supported:

```
template<  
    typename SocketService,  
    typename Fn>  
void async_accept(  
    basic_socket< protocol_type, SocketService > & peer,  
    endpoint_type & peer_endpoint,  
    Fn handler);
```

The `async_accept()` signature accepts an Invocable callback as the last argument. The preceding arguments are inputs needed to produce the result.

When the Invocable callback is an argument, there is a need to be able to distinguish the input arguments from the Invocable callback argument. C++ has no way to universally inspect the arguments of the function overload

that would be selected by a particular set of arguments. There are proposals that would help, and until that support is available, distinguishing the Invocable callback and input arguments must be done manually or by convention. There is also a need to supply the input arguments in one code path and the Invocable callback in another. Separating the Invocable callback argument out and supplying the Invocable callback arg later requires function binding, on top of the ability to distinguish the input and Invocable callback arguments (for now the options for this are manual and by-convention).

Taking the Invocable callback argument as the last argument affects valid signatures for and overloading of functions as well. Having a fixed last argument, requires adding overloads when defaulting the values of input arguments. Having a fixed last argument, requires that the overloads be constrainable so that the Invocable callback can be reliably distinguished from the input arguments. Having a fixed last argument, prevents using variadic input arguments, which is why `std::visit` places the Invocable callback as the first argument.

10.2.2 return value style

Using the return value from a function to separate the input and callback arguments allows the operation and the attachment of a callback to be deferred. Using a return value produces very different tradeoffs than when mixing callback and inputs together into the function arguments.

The Sender concept defined in [P1660R0] has been gaining support in SG1 recently. A signature for the `async_accept()` example might change to look something like:

```
template<
    typename SocketService
auto async_accept(
    basic_socket< protocol_type, SocketService > & peer,
    endpoint_type & peer_endpoint);
```

When a callback is passed to a return value, there is no need to be able to distinguish the input arguments from the callback argument. When a callback is passed to a return value, the input arguments are the only arguments to the function and can be supplied in one code path and another code path can supply a callback to the return value of the function.

Returning a value that allows a callback to be attached later has no affect on valid signatures for, and overloading of, functions. Functions with overloads and variadic arguments behave no differently from any other function when returning a value allows a callback to be attached later.

10.2.3 Networking TS style

The Networking TS [N4771] uses the [completion handler style](#) described in [N4045] to overload the callback argument to async functions like `async_accept()` (Accepting an Invocable callback indicates that the Networking TS style subsumes the [function argument style](#) and the [function argument style](#) using the last argument). On the other hand, if the last argument [function argument style](#) is a completion handler, then the completion handler implements an Invocable callback [function argument style](#) that matches the completion signature for the function and delivers the result to that implementation. `async_accept()`'s signature, as defined in [N4771]:

```
template<
    typename SocketService,
    typename AcceptHandler>
void-or-deduced async_accept(
    basic_socket< protocol_type, SocketService > & peer,
    endpoint_type & peer_endpoint,
    AcceptHandler handler);
```

The completion handler described in [N4045] and used in the Networking TS [N4771] has a limitation that was recently addressed in Boost.Asio. `async_result<>::initiate` ([Boost.Asio revision history](#), [github](#)) was added

recently to allow the completion handler to describe a return value that would be allowed to defer the start of the async function and allow the callback to be attached later. The returned value would also be allowed to transfer the storage of the state for the operation to the caller, by storing that state in the returned value.

Note: at the time this note was written, there is no paper proposing to add `async_initiate` to the Networking TS.

`async_initiate` enables the start of the operation to be Deferred, which can be used to reduce composition and runtime overhead. Without deferral it is complicated to write a generic retry function that starts an operation over again when it fails, because the async function has to be called with arguments and the callback/completion handler.

`async_initiate` enables State transfer from the operation to the caller, which allows coroutine support that stores the state for the operation on the calling coroutine frame avoiding additional allocations.

The Networking TS [N4771], when updated to include `async_initiate`, will allow a completion handler to support deferral of the operation and separate the attachment of a callback from supplying the input arguments (`async_initiate` allows the creation of completion handlers that enables the Networking TS style to emulate the [return value style](#)). Emulating the [return value style](#) requires manually or by convention inserting the right completion handler into the correct argument, which means that even when emulating the [return value style](#) the Networking TS style inherits the affects of having a fixed last argument from [function argument style](#).

11 Credits

This paper was influenced by hosts of people over decades.

- **Marc Barbour** and **Mark Lawrence** were fundamental to Kirk's first attempt to design more regular callbacks in a COM environment.
- **Aaron Lahman** was involved in that first attempt as well and introduced Kirk to the Reactive-Extensions libraries because he saw the similarity.
- **Erik Meijer** and his team took a very different path to arrive at a destination that resonated strongly with Kirk's goals
- *Microsoft Open Technologies Inc.* led by **Jean Paoli**, encouraged and supported Kirk's subsequent investment in finishing Aaron's C++ Rx prototype and then rewriting it to shift from interfaces to compile-time polymorphism.
- **Ben Christensen** drove changes to RxJava and his communication around those changes affected the design Kirk chose for rxcpp
- **Grigorii Chudnov**, **Valery Kopylov** and all the other amazing contributors to rxcpp over the years
- **Eric Niebler**, **Lee Howes** and **Lewis Baker** who more than anyone else contributed to the content of the motivation section of this paper
- *CppCon*, *CppNow*, *CppRussia* and *CERN* (and the people behind those including; **Jon Kalb**, **Bryce Adelstein-Lebach**, **Sergey Platonov**, **Axel Naumann**) for all the opportunities to communicate the vision for callbacks in C++
- **Gor Nishanov** for the excellent coroutines in C++20 and the shout-outs and support for rxcpp over the years.

12 Appendices

12.1 `future_from` implementation

This section provides contrast between the representations proposed here with the ~20 pages of implementation in [use_future.hpp](#) which is proposed by the Networking TS [N4771].

12.1.1 support std::promise as a receiver

Source: <https://github.com/facebook/folly/blob/master/folly/experimental/pushmi/receiver/primitives.h>

```
//  
// add support for std::promise externally  
//  
  
// std::promise does not support the done signal.  
// either set_value or set_error must be called  
template <class T>  
void set_done(std::promise<T>&) noexcept {}  
  
template <class T>  
void set_error(std::promise<T>& p, std::exception_ptr e) noexcept {  
    p.set_exception(std::move(e));  
}  
  
template <class T, class E>  
void set_error(std::promise<T>& p, E e) noexcept {  
    p.set_exception(std::make_exception_ptr(std::move(e)));  
}  
  
PUSHMI_TEMPLATE(class T, class U)  
(requires ConvertibleTo<U, T>) //  
void set_value(std::promise<T>& p, U&& u) //  
    noexcept(noexcept(p.set_value((U &&) u))) {  
    p.set_value((U &&) u);  
}  
  
inline void set_value(std::promise<void>& p) //  
    noexcept(noexcept(p.set_value())) {  
    p.set_value();  
}
```

12.1.2 implement future_from

Source: <https://github.com/facebook/folly/blob/master/folly/experimental/pushmi/receiver/receiver.h>

```
PUSHMI_TEMPLATE (class T, class In)  
    (requires SenderTo<In, std::promise<T>> && SingleSender<In>)  
std::future<T> future_from(In&& in) {  
    std::promise<T> p;  
    auto result = p.get_future();  
    submit((In&&) in, std::move(p));  
    return result;  
}  
  
PUSHMI_TEMPLATE (class In)  
    (requires SenderTo<In, std::promise<void>> && SingleSender<In>)  
std::future<void> future_from(In&& in) {  
    std::promise<void> p;  
    auto result = p.get_future();  
    submit((In&&) in, std::move(p));  
}
```

```

return result;
}

```

12.2 use_future completion handler implementation

The `use_future` completion handler implementation from the Networking TS [N4771] is an example of what is required to build and use completion handlers that need to distinguish error and value arguments in the completion signature. `use_awaitable`, `use_pipe` and `use_sender` would be similar.

There are two files involved in implementing the semantics defined in this table from [N4771]

Table 17: [N4771] Table 10 - `async_result<use_future_t, Result(Args...)>` semantics

N	U0	R::return_type	F::operator() effects
0		<code>future<void></code>	None.
1	<code>error_code</code>	<code>future<void></code>	If <code>a0</code> evaluates to true, atomically stores the exception pointer produced by <code>make_exception_ptr(system_error(a0))</code> in the shared state.
1	<code>exception_ptr</code>	<code>future<void></code>	If <code>a0</code> is non-null, atomically stores the exception pointer <code>a0</code> in the shared state.
1	all other types	<code>future<U0></code>	Atomically stores <code>forward<A0>(a0)</code> in the shared state.
2	<code>error_code</code>	<code>future<U1></code>	If <code>a0</code> evaluates to true, atomically stores the exception pointer produced by <code>make_exception_ptr(system_error(a0))</code> in the shared state; otherwise, atomically stores <code>forward(a1)</code> in the shared state.
2	<code>exception_ptr</code>	<code>future<U1></code>	If <code>a0</code> is non-null, atomically stores the exception pointer in the shared state; otherwise, atomically stores <code>forward<A1>(a1)</code> in the shared state.
2	all other types	<code>future<tuple<U0, U1>></code>	Atomically stores <code>forward_as_tuple(forward<A0>(a0), forward<A1>(a1))</code> in the shared state.
2	<code>error_code</code>	<code>future<tuple<U1, ..., UN-1>></code>	If <code>a0</code> evaluates to true, atomically stores the exception pointer produced by <code>make_exception_ptr(system_error(a0))</code> in the shared state; otherwise, atomically stores <code>forward_as_tuple(forward<A1>(a1), ..., forward<AN-1>(aN-1))</code> in the shared state.

N	U0	R::return_type	F::operator() effects
>2	exception_ptr	future<tuple<U1, ... , UN-1>>	If a0 is non-null, atomically stores the exception pointer in the shared state; otherwise, atomically stores <code>forward_as_tuple(forward<A1>(a1), ... , forward<AN-1>(aN-1))</code> in the shared state.
>2	all other types	future<tuple<U0, ... , UN-1>>	Atomically stores <code>forward_as_tuple(forward<A0>(a0), ... , forward<AN-1>(aN-1))</code> in the shared state.

These semantics attempt to split the error and values arguments apart and then route them to the `set_value()` and `set_exception()` methods on a promise.

Obvious limitations:

- only `exception_ptr` & `error_code` are recognized as error types and only when one of them is the first argument.
- partial success requires an api to specify a completion signature with two error arguments, like: `void(error_code, error_code, T...)` because the first error argument will be forwarded to `set_exception()` in which case `set_value()` will not be called.

12.2.1 use_future.hpp

Source: https://github.com/chriskohlhoff/networking-ts-impl/blob/master/include/experimental/__net_ts/use_future.hpp

```
//
// use_future.hpp
// ~~~~~
//
// Copyright (c) 2003-2019 Christopher M. Kohlhoff (chris at kohlhoff dot com)
//
// Distributed under the Boost Software License, Version 1.0. (See accompanying
// file LICENSE_1_0.txt or copy at http://www.boost.org/LICENSE_1_0.txt)
//

#ifndef NET_TS_USE_FUTURE_HPP
#define NET_TS_USE_FUTURE_HPP

#if defined(_MSC_VER) && (_MSC_VER >= 1200)
# pragma once
#endif // defined(_MSC_VER) && (_MSC_VER >= 1200)

#include <experimental/__net_ts/detail/config.hpp>
#include <experimental/__net_ts/detail/future.hpp>

#if defined(NET_TS_HAS_STD_FUTURE_CLASS) \
    || defined(GENERATING_DOCUMENTATION)

#include <memory>
#include <experimental/__net_ts/detail/type_traits.hpp>
```

```

#include <experimental/_net_ts/detail/push_options.hpp>

namespace std {
namespace experimental {
namespace net {
inline namespace v1 {
namespace detail {

template <typename Function, typename Allocator>
class packaged_token;

template <typename Function, typename Allocator, typename Result>
class packaged_handler;

} // namespace detail

/// Class used to specify that an asynchronous operation should return a future.
/**
 * The use_future_t class is used to indicate that an asynchronous operation
 * should return a std::future object. A use_future_t object may be passed as a
 * handler to an asynchronous operation, typically using the special value @c
 * std::experimental::net::v1::use_future. For example:
 *
 * @code std::future<std::size_t> my_future
 *   = my_socket.async_read_some(my_buffer, std::experimental::net::use_future); @endcode
 *
 * The initiating function (async_read_some in the above example) returns a
 * future that will receive the result of the operation. If the operation
 * completes with an error_code indicating failure, it is converted into a
 * system_error and passed back to the caller via the future.
 */

template <typename Allocator = std::allocator<void> >
class use_future_t
{
public:
/// The allocator type. The allocator is used when constructing the
/// @c std::promise object for a given asynchronous operation.
    typedef Allocator allocator_type;

/// Construct using default-constructed allocator.
    NET_TS_CONSTEXPR use_future_t()
    {
    }

/// Construct using specified allocator.
    explicit use_future_t(const Allocator& allocator)
        : allocator_(allocator)
    {
    }

/// Specify an alternate allocator.
    template <typename OtherAllocator>
    use_future_t<OtherAllocator> rebind(const OtherAllocator& allocator) const

```

```

{
    return use_future_t<OtherAllocator>(allocator);
}

/// Obtain allocator.
allocator_type get_allocator() const
{
    return allocator_;
}

/// Wrap a function object in a packaged task.
/**
 * The @c package function is used to adapt a function object as a packaged
 * task. When this adapter is passed as a completion token to an asynchronous
 * operation, the result of the function object is returned via a std::future.
 *
 * @par Example
 *
 * @code std::future<std::size_t> fut =
 *     my_socket.async_read_some(buffer,
 *         use_future([](std::error_code ec, std::size_t n)
 *             {
 *                 return ec ? 0 : n;
 *             }));
 * ...
 * std::size_t n = fut.get(); @endcode
 */
template <typename Function>
#if defined(GENERATING_DOCUMENTATION)
    unspecified
#else // defined(GENERATING_DOCUMENTATION)
    detail::packaged_token<typename decay<Function>::type, Allocator>
#endif // defined(GENERATING_DOCUMENTATION)
    operator()(NET_TS_MOVE_ARG(Function) f) const;

private:
    /// Helper type to ensure that use_future can be constexpr default-constructed
    /// even when std::allocator<void> can't be.
    struct std_allocator_void
    {
        NET_TS_CONSTEXPR std_allocator_void()
        {
        }

        operator std::allocator<void>() const
        {
            return std::allocator<void>();
        }
    };

    typename conditional<
        is_same<std::allocator<void>, Allocator>::value,
        std_allocator_void, Allocator>::type allocator_;

```

```
};

/// A special value, similar to std::nothrow.
/**
 * See the documentation for std::experimental::net::v1::use_future_t for a usage example.
 */
#if defined(NET_TS_HAS_CONSTEXPR) || defined(GENERATING_DOCUMENTATION)
constexpr use_future_t<> use_future;
#elif defined(NET_TS_MSVC)
__declspec(selectany) use_future_t<> use_future;
#endif

} // inline namespace v1
} // namespace net
} // namespace experimental
} // namespace std

#include <experimental/___net_ts/detail/pop_options.hpp>

#include <experimental/___net_ts/impl/use_future.hpp>

#endif // defined(NET_TS_HAS_STD_FUTURE_CLASS)
      // || defined(GENERATING_DOCUMENTATION)

#endif // NET_TS_USE_FUTURE_HPP
```

12.2.2 impl/use_future.hpp

Source: https://github.com/chriskohlhoff/networking-ts-impl/blob/master/include/experimental/___net_ts/impl/use_future.hpp

```
//
// impl/use_future.hpp
// ~~~~~
//
// Copyright (c) 2003-2019 Christopher M. Kohlhoff (chris at kohlhoff dot com)
//
// Distributed under the Boost Software License, Version 1.0. (See accompanying
// file LICENSE_1_0.txt or copy at http://www.boost.org/LICENSE_1_0.txt)
//

#ifndef NET_TS_IMPL_USE_FUTURE_HPP
#define NET_TS_IMPL_USE_FUTURE_HPP

#if defined(_MSC_VER) && (_MSC_VER >= 1200)
# pragma once
#endif // defined(_MSC_VER) && (_MSC_VER >= 1200)

#include <experimental/___net_ts/detail/config.hpp>
#include <tuple>
#include <experimental/___net_ts/async_result.hpp>
#include <experimental/___net_ts/detail/memory.hpp>
#include <system_error>
```

```

#include <experimental/___net_ts/packaged_task.hpp>
#include <system_error>
#include <experimental/___net_ts/system_executor.hpp>

#include <experimental/___net_ts/detail/push_options.hpp>

namespace std {
namespace experimental {
namespace net {
inline namespace v1 {
namespace detail {

#if defined(NET_TS_HAS_VARIADIC_TEMPLATES)

template <typename T, typename F, typename... Args>
inline void promise_invoke_and_set(std::promise<T>& p,
    F& f, NET_TS_MOVE_ARG(Args)... args)
{
#if !defined(NET_TS_NO_EXCEPTIONS)
    try
#endif // !defined(NET_TS_NO_EXCEPTIONS)
    {
        p.set_value(f(NET_TS_MOVE_CAST(Args)(args)...));
    }
#if !defined(NET_TS_NO_EXCEPTIONS)
    catch (...)
    {
        p.set_exception(std::current_exception());
    }
#endif // !defined(NET_TS_NO_EXCEPTIONS)
}

template <typename F, typename... Args>
inline void promise_invoke_and_set(std::promise<void>& p,
    F& f, NET_TS_MOVE_ARG(Args)... args)
{
#if !defined(NET_TS_NO_EXCEPTIONS)
    try
#endif // !defined(NET_TS_NO_EXCEPTIONS)
    {
        f(NET_TS_MOVE_CAST(Args)(args)...);
        p.set_value();
    }
#if !defined(NET_TS_NO_EXCEPTIONS)
    catch (...)
    {
        p.set_exception(std::current_exception());
    }
#endif // !defined(NET_TS_NO_EXCEPTIONS)
}

#else // defined(NET_TS_HAS_VARIADIC_TEMPLATES)

```

```

template <typename T, typename F>
inline void promise_invoke_and_set(std::promise<T>& p, F& f)
{
#if !defined(NET_TS_NO_EXCEPTIONS)
    try
#endif // !defined(NET_TS_NO_EXCEPTIONS)
    {
        p.set_value(f());
    }
#if !defined(NET_TS_NO_EXCEPTIONS)
    catch (...)
    {
        p.set_exception(std::current_exception());
    }
#endif // !defined(NET_TS_NO_EXCEPTIONS)
}

template <typename F, typename Args>
inline void promise_invoke_and_set(std::promise<void>& p, F& f)
{
#if !defined(NET_TS_NO_EXCEPTIONS)
    try
#endif // !defined(NET_TS_NO_EXCEPTIONS)
    {
        f();
        p.set_value();
    }
#if !defined(NET_TS_NO_EXCEPTIONS)
    catch (...)
    {
        p.set_exception(std::current_exception());
    }
#endif // !defined(NET_TS_NO_EXCEPTIONS)
}

#if defined(NET_TS_NO_EXCEPTIONS)

#define NET_TS_PRIVATE_PROMISE_INVOKE_DEF(n) \
    template <typename T, typename F, NET_TS_VARIADIC_TPARAMS(n)> \
    inline void promise_invoke_and_set(std::promise<T>& p, \
        F& f, NET_TS_VARIADIC_MOVE_PARAMS(n)) \
    { \
        p.set_value(f(NET_TS_VARIADIC_MOVE_ARGS(n))); \
    } \
    \
    template <typename F, NET_TS_VARIADIC_TPARAMS(n)> \
    inline void promise_invoke_and_set(std::promise<void>& p, \
        F& f, NET_TS_VARIADIC_MOVE_PARAMS(n)) \
    { \
        f(NET_TS_VARIADIC_MOVE_ARGS(n)); \
        p.set_value(); \
    } \
    /**/
NET_TS_VARIADIC_GENERATE(NET_TS_PRIVATE_PROMISE_INVOKE_DEF)

```

```

#undef NET_TS_PRIVATE_PROMISE_INVOKE_DEF

#else // defined(NET_TS_NO_EXCEPTIONS)

#define NET_TS_PRIVATE_PROMISE_INVOKE_DEF(n) \
    template <typename T, typename F, NET_TS_VARIADIC_TPARAMS(n)> \
    inline void promise_invoke_and_set(std::promise<T>& p, \
        F& f, NET_TS_VARIADIC_MOVE_PARAMS(n)) \
    { \
        try \
        { \
            p.set_value(f(NET_TS_VARIADIC_MOVE_ARGS(n))); \
        } \
        catch (...) \
        { \
            p.set_exception(std::current_exception()); \
        } \
    } \
    \
    template <typename F, NET_TS_VARIADIC_TPARAMS(n)> \
    inline void promise_invoke_and_set(std::promise<void>& p, \
        F& f, NET_TS_VARIADIC_MOVE_PARAMS(n)) \
    { \
        try \
        { \
            f(NET_TS_VARIADIC_MOVE_ARGS(n)); \
            p.set_value(); \
        } \
        catch (...) \
        { \
            p.set_exception(std::current_exception()); \
        } \
    } \
    /**/
    NET_TS_VARIADIC_GENERATE(NET_TS_PRIVATE_PROMISE_INVOKE_DEF)
#undef NET_TS_PRIVATE_PROMISE_INVOKE_DEF

#endif // defined(NET_TS_NO_EXCEPTIONS)

#endif // defined(NET_TS_HAS_VARIADIC_TEMPLATES)

// A function object adapter to invoke a nullary function object and capture
// any exception thrown into a promise.
template <typename T, typename F>
class promise_invoker
{
public:
    promise_invoker(const shared_ptr<std::promise<T> >& p,
        NET_TS_MOVE_ARG(F) f)
        : p_(p), f_(NET_TS_MOVE_CAST(F)(f))
    {
    }

    void operator()()

```

```

    {
#if !defined(NET_TS_NO_EXCEPTIONS)
        try
#endif // !defined(NET_TS_NO_EXCEPTIONS)
        {
            f_();
        }
#if !defined(NET_TS_NO_EXCEPTIONS)
        catch (...)
        {
            p_>set_exception(std::current_exception());
        }
#endif // !defined(NET_TS_NO_EXCEPTIONS)
    }

private:
    shared_ptr<std::promise<T> > p_;
    typename decay<F>::type f_;
};

// An executor that adapts the system_executor to capture any exeption thrown
// by a submitted function object and save it into a promise.
template <typename T>
class promise_executor
{
public:
    explicit promise_executor(const shared_ptr<std::promise<T> >& p)
        : p_(p)
    {
    }

    execution_context& context() const NET_TS_NOEXCEPT
    {
        return system_executor().context();
    }

    void on_work_started() const NET_TS_NOEXCEPT {}
    void on_work_finished() const NET_TS_NOEXCEPT {}

    template <typename F, typename A>
    void dispatch(NET_TS_MOVE_ARG(F) f, const A&) const
    {
        promise_invoker<T, F>(p_, NET_TS_MOVE_CAST(F)(f))();
    }

    template <typename F, typename A>
    void post(NET_TS_MOVE_ARG(F) f, const A& a) const
    {
        system_executor().post(
            promise_invoker<T, F>(p_, NET_TS_MOVE_CAST(F)(f)), a);
    }

    template <typename F, typename A>

```

```

void defer(NET_TS_MOVE_ARG(F) f, const A& a) const
{
    system_executor().defer(
        promise_invoker<T, F>(p_, NET_TS_MOVE_CAST(F)(f)), a);
}

friend bool operator==(const promise_executor& a,
    const promise_executor& b) NET_TS_NOEXCEPT
{
    return a.p_ == b.p_;
}

friend bool operator!=(const promise_executor& a,
    const promise_executor& b) NET_TS_NOEXCEPT
{
    return a.p_ != b.p_;
}

private:
    shared_ptr<std::promise<T> > p_;
};

// The base class for all completion handlers that create promises.
template <typename T>
class promise_creator
{
public:
    typedef promise_executor<T> executor_type;

    executor_type get_executor() const NET_TS_NOEXCEPT
    {
        return executor_type(p_);
    }

    typedef std::future<T> future_type;

    future_type get_future()
    {
        return p_->get_future();
    }

protected:
    template <typename Allocator>
    void create_promise(const Allocator& a)
    {
        NET_TS_REBIND_ALLOC(Allocator, char) b(a);
        p_ = std::allocate_shared<std::promise<T>>(b, std::allocator_arg, b);
    }

    shared_ptr<std::promise<T> > p_;
};

// For completion signature void().

```

```

class promise_handler_0
: public promise_creator<void>
{
public:
    void operator()()
    {
        this->p_->set_value();
    }
};

// For completion signature void(error_code).
class promise_handler_ec_0
: public promise_creator<void>
{
public:
    void operator()(const std::error_code& ec)
    {
        if (ec)
        {
            this->p_->set_exception(
                std::make_exception_ptr(
                    std::system_error(ec)));
        }
        else
        {
            this->p_->set_value();
        }
    }
};

// For completion signature void(exception_ptr).
class promise_handler_ex_0
: public promise_creator<void>
{
public:
    void operator()(const std::exception_ptr& ex)
    {
        if (ex)
        {
            this->p_->set_exception(ex);
        }
        else
        {
            this->p_->set_value();
        }
    }
};

// For completion signature void(T).
template <typename T>
class promise_handler_1
: public promise_creator<T>
{
public:

```

```

template <typename Arg>
void operator()(NET_TS_MOVE_ARG(Arg) arg)
{
    this->p_->set_value(NET_TS_MOVE_CAST(Arg)(arg));
}
};

// For completion signature void(error_code, T).
template <typename T>
class promise_handler_ec_1
    : public promise_creator<T>
{
public:
    template <typename Arg>
    void operator()(const std::error_code& ec,
        NET_TS_MOVE_ARG(Arg) arg)
    {
        if (ec)
        {
            this->p_->set_exception(
                std::make_exception_ptr(
                    std::system_error(ec)));
        }
        else
            this->p_->set_value(NET_TS_MOVE_CAST(Arg)(arg));
    }
};

// For completion signature void(exception_ptr, T).
template <typename T>
class promise_handler_ex_1
    : public promise_creator<T>
{
public:
    template <typename Arg>
    void operator()(const std::exception_ptr& ex,
        NET_TS_MOVE_ARG(Arg) arg)
    {
        if (ex)
            this->p_->set_exception(ex);
        else
            this->p_->set_value(NET_TS_MOVE_CAST(Arg)(arg));
    }
};

// For completion signature void(T1, ..., Tn);
template <typename T>
class promise_handler_n
    : public promise_creator<T>
{
public:
#ifdef NET_TS_HAS_VARIADIC_TEMPLATES
    template <typename... Args>

```

```

void operator()(NET_TS_MOVE_ARG(Args)... args)
{
    this->p_->set_value(
        std::forward_as_tuple(
            NET_TS_MOVE_CAST(Args)(args)...));
}

#else // defined(NET_TS_HAS_VARIADIC_TEMPLATES)

#define NET_TS_PRIVATE_CALL_OP_DEF(n) \
    template <NET_TS_VARIADIC_TPARAMS(n)> \
    void operator()(NET_TS_VARIADIC_MOVE_PARAMS(n)) \
    {\
        this->p_->set_value( \
            std::forward_as_tuple( \
                NET_TS_VARIADIC_MOVE_ARGS(n))); \
    } \
    /**/
    NET_TS_VARIADIC_GENERATE(NET_TS_PRIVATE_CALL_OP_DEF)
#undef NET_TS_PRIVATE_CALL_OP_DEF

#endif // defined(NET_TS_HAS_VARIADIC_TEMPLATES)
};

// For completion signature void(error_code, T1, ..., Tn);
template <typename T>
class promise_handler_ec_n
    : public promise_creator<T>
{
public:
#if defined(NET_TS_HAS_VARIADIC_TEMPLATES)

    template <typename... Args>
    void operator()(const std::error_code& ec,
        NET_TS_MOVE_ARG(Args)... args)
    {
        if (ec)
        {
            this->p_->set_exception(
                std::make_exception_ptr(
                    std::system_error(ec)));
        }
        else
        {
            this->p_->set_value(
                std::forward_as_tuple(
                    NET_TS_MOVE_CAST(Args)(args)...));
        }
    }

#else // defined(NET_TS_HAS_VARIADIC_TEMPLATES)

#define NET_TS_PRIVATE_CALL_OP_DEF(n) \

```

```

template <NET_TS_VARIADIC_TP_PARAMS(n)> \
void operator()(const std::error_code& ec, \
    NET_TS_VARIADIC_MOVE_PARAMS(n)) \
{ \
    if (ec) \
    { \
        this->p_->set_exception( \
            std::make_exception_ptr( \
                std::system_error(ec))); \
    } \
    else \
    { \
        this->p_->set_value( \
            std::forward_as_tuple( \
                NET_TS_VARIADIC_MOVE_ARGS(n))); \
    } \
} \
/**/
NET_TS_VARIADIC_GENERATE(NET_TS_PRIVATE_CALL_OP_DEF)
#undef NET_TS_PRIVATE_CALL_OP_DEF

#endif // defined(NET_TS_HAS_VARIADIC_TEMPLATES)
};

// For completion signature void(exception_ptr, T1, ..., Tn);
template <typename T>
class promise_handler_ex_n
    : public promise_creator<T>
{
public:
#if defined(NET_TS_HAS_VARIADIC_TEMPLATES)

    template <typename... Args>
    void operator()(const std::exception_ptr& ex,
        NET_TS_MOVE_ARG(Args)... args)
    {
        if (ex)
            this->p_->set_exception(ex);
        else
        {
            this->p_->set_value(
                std::forward_as_tuple(
                    NET_TS_MOVE_CAST(Args)(args)...));
        }
    }
}

#else // defined(NET_TS_HAS_VARIADIC_TEMPLATES)

#define NET_TS_PRIVATE_CALL_OP_DEF(n) \
    template <NET_TS_VARIADIC_TP_PARAMS(n)> \
    void operator()(const std::exception_ptr& ex, \
        NET_TS_VARIADIC_MOVE_PARAMS(n)) \
    { \

```

```

    if (ex) \
        this->p_->set_exception(ex); \
    else \
    { \
        this->p_->set_value( \
            std::forward_as_tuple( \
                NET_TS_VARIADIC_MOVE_ARGS(n)); \
        ) \
    } \
} \
/**/
NET_TS_VARIADIC_GENERATE(NET_TS_PRIVATE_CALL_OP_DEF)
#undef NET_TS_PRIVATE_CALL_OP_DEF

#endif // defined(NET_TS_HAS_VARIADIC_TEMPLATES)
};

// Helper template to choose the appropriate concrete promise handler
// implementation based on the supplied completion signature.
template <typename> class promise_handler_selector;

template <>
class promise_handler_selector<void()>
    : public promise_handler_0 {};

template <>
class promise_handler_selector<void(std::error_code)>
    : public promise_handler_ec_0 {};

template <>
class promise_handler_selector<void(std::exception_ptr)>
    : public promise_handler_ex_0 {};

template <typename Arg>
class promise_handler_selector<void(Arg)>
    : public promise_handler_1<Arg> {};

template <typename Arg>
class promise_handler_selector<void(std::error_code, Arg)>
    : public promise_handler_ec_1<Arg> {};

template <typename Arg>
class promise_handler_selector<void(std::exception_ptr, Arg)>
    : public promise_handler_ex_1<Arg> {};

#if defined(NET_TS_HAS_VARIADIC_TEMPLATES)

template <typename... Arg>
class promise_handler_selector<void(Arg...)>
    : public promise_handler_n<std::tuple<Arg...> > {};

template <typename... Arg>
class promise_handler_selector<void(std::error_code, Arg...)>
    : public promise_handler_ec_n<std::tuple<Arg...> > {};

```

```

template <typename... Arg>
class promise_handler_selector<void(std::exception_ptr, Arg...)>
    : public promise_handler_ex_n<std::tuple<Arg...> > {};

#else // defined(NET_TS_HAS_VARIADIC_TEMPLATES)

#define NET_TS_PRIVATE_PROMISE_SELECTOR_DEF(n) \
    template <typename Arg, NET_TS_VARIADIC_TPARAMS(n)> \
    class promise_handler_selector< \
        void(Arg, NET_TS_VARIADIC_TARGS(n))> \
        : public promise_handler_n< \
            std::tuple<Arg, NET_TS_VARIADIC_TARGS(n)> > {}; \
    \
    template <typename Arg, NET_TS_VARIADIC_TPARAMS(n)> \
    class promise_handler_selector< \
        void(std::error_code, Arg, NET_TS_VARIADIC_TARGS(n))> \
        : public promise_handler_ec_n< \
            std::tuple<Arg, NET_TS_VARIADIC_TARGS(n)> > {}; \
    \
    template <typename Arg, NET_TS_VARIADIC_TPARAMS(n)> \
    class promise_handler_selector< \
        void(std::exception_ptr, Arg, NET_TS_VARIADIC_TARGS(n))> \
        : public promise_handler_ex_n< \
            std::tuple<Arg, NET_TS_VARIADIC_TARGS(n)> > {}; \
    /**/
    NET_TS_VARIADIC_GENERATE(NET_TS_PRIVATE_PROMISE_SELECTOR_DEF)
#undef NET_TS_PRIVATE_PROMISE_SELECTOR_DEF

#endif // defined(NET_TS_HAS_VARIADIC_TEMPLATES)

// Completion handlers produced from the use_future completion token, when not
// using use_future::operator().
template <typename Signature, typename Allocator>
class promise_handler
    : public promise_handler_selector<Signature>
{
public:
    typedef Allocator allocator_type;
    typedef void result_type;

    promise_handler(use_future_t<Allocator> u)
        : allocator_(u.get_allocator())
    {
        this->create_promise(allocator_);
    }

    allocator_type get_allocator() const NET_TS_NOEXCEPT
    {
        return allocator_;
    }

private:
    Allocator allocator_;

```

```

};

template <typename Function, typename Signature, typename Allocator>
inline void networking_ts_handler_invoke(Function& f,
    promise_handler<Signature, Allocator>* h)
{
    typename promise_handler<Signature, Allocator>::executor_type
        ex(h->get_executor());
    ex.dispatch(NET_TS_MOVE_CAST(Function)(f), std::allocator<void>());
}

template <typename Function, typename Signature, typename Allocator>
inline void networking_ts_handler_invoke(const Function& f,
    promise_handler<Signature, Allocator>* h)
{
    typename promise_handler<Signature, Allocator>::executor_type
        ex(h->get_executor());
    ex.dispatch(f, std::allocator<void>());
}

// Helper base class for async_result specialisation.
template <typename Signature, typename Allocator>
class promise_async_result
{
public:
    typedef promise_handler<Signature, Allocator> completion_handler_type;
    typedef typename completion_handler_type::future_type return_type;

    explicit promise_async_result(completion_handler_type& h)
        : future_(h.get_future())
    {
    }

    return_type get()
    {
        return NET_TS_MOVE_CAST(return_type)(future_);
    }

private:
    return_type future_;
};

// Return value from use_future::operator().
template <typename Function, typename Allocator>
class packaged_token
{
public:
    packaged_token(Function f, const Allocator& a)
        : function_(NET_TS_MOVE_CAST(Function)(f)),
          allocator_(a)
    {
    }
}

```

```

//private:
    Function function_;
    Allocator allocator_;
};

// Completion handlers produced from the use_future completion token, when
// using use_future::operator().
template <typename Function, typename Allocator, typename Result>
class packaged_handler
    : public promise_creator<Result>
{
public:
    typedef Allocator allocator_type;
    typedef void result_type;

    packaged_handler(packaged_token<Function, Allocator> t)
        : function_(NET_TS_MOVE_CAST(Function)(t.function_)),
          allocator_(t.allocator_)
    {
        this->create_promise(allocator_);
    }

    allocator_type get_allocator() const NET_TS_NOEXCEPT
    {
        return allocator_;
    }
};

#if defined(NET_TS_HAS_VARIADIC_TEMPLATES)

    template <typename... Args>
    void operator()(NET_TS_MOVE_ARG(Args)... args)
    {
        (promise_invoke_and_set)(*this->p_,
            function_, NET_TS_MOVE_CAST(Args)(args)...);
    }

#else // defined(NET_TS_HAS_VARIADIC_TEMPLATES)

    void operator()()
    {
        (promise_invoke_and_set)(*this->p_, function_);
    }

#define NET_TS_PRIVATE_CALL_OP_DEF(n) \
    template <NET_TS_VARIADIC_TPARAMS(n)> \
    void operator()(NET_TS_VARIADIC_MOVE_PARAMS(n)) \
    { \
        (promise_invoke_and_set)(*this->p_, \
            function_, NET_TS_VARIADIC_MOVE_ARGS(n)); \
    } \
    /**/
    NET_TS_VARIADIC_GENERATE(NET_TS_PRIVATE_CALL_OP_DEF)
#undef NET_TS_PRIVATE_CALL_OP_DEF

```

```

#endif // defined(NET_TS_HAS_VARIADIC_TEMPLATES)

private:
    Function function_;
    Allocator allocator_;
};

template <typename Function,
          typename Function1, typename Allocator, typename Result>
inline void networking_ts_handler_invoke(Function& f,
    packaged_handler<Function1, Allocator, Result>* h)
{
    typename packaged_handler<Function1, Allocator, Result>::executor_type
        ex(h->get_executor());
    ex.dispatch(NET_TS_MOVE_CAST(Function)(f), std::allocator<void>());
}

template <typename Function,
          typename Function1, typename Allocator, typename Result>
inline void networking_ts_handler_invoke(const Function& f,
    packaged_handler<Function1, Allocator, Result>* h)
{
    typename packaged_handler<Function1, Allocator, Result>::executor_type
        ex(h->get_executor());
    ex.dispatch(f, std::allocator<void>());
}

// Helper base class for async_result specialisation.
template <typename Function, typename Allocator, typename Result>
class packaged_async_result
{
public:
    typedef packaged_handler<Function, Allocator, Result> completion_handler_type;
    typedef typename completion_handler_type::future_type return_type;

    explicit packaged_async_result(completion_handler_type& h)
        : future_(h.get_future())
    {
    }

    return_type get()
    {
        return NET_TS_MOVE_CAST(return_type)(future_);
    }

private:
    return_type future_;
};

} // namespace detail

template <typename Allocator> template <typename Function>
inline detail::packaged_token<typename decay<Function>::type, Allocator>

```

```

use_future_t<Allocator>::operator()(NET_TS_MOVE_ARG(Function) f) const
{
    return detail::packaged_token<typename decay<Function>::type, Allocator>(
        NET_TS_MOVE_CAST(Function)(f), allocator_);
}

#if !defined(GENERATING_DOCUMENTATION)

#if defined(NET_TS_HAS_VARIADIC_TEMPLATES)

template <typename Allocator, typename Result, typename... Args>
class async_result<use_future_t<Allocator>, Result(Args...)>
    : public detail::promise_async_result<
        void(typename decay<Args>::type...), Allocator>
{
public:
    explicit async_result(
        typename detail::promise_async_result<void(typename decay<Args>::type...),
            Allocator>::completion_handler_type& h)
        : detail::promise_async_result<
            void(typename decay<Args>::type...), Allocator>(h)
    {
    }
};

template <typename Function, typename Allocator,
    typename Result, typename... Args>
class async_result<detail::packaged_token<Function, Allocator>, Result(Args...)>
    : public detail::packaged_async_result<Function, Allocator,
        typename result_of<Function(Args...)>::type>
{
public:
    explicit async_result(
        typename detail::packaged_async_result<Function, Allocator,
            typename result_of<Function(Args...)>::type>::completion_handler_type& h)
        : detail::packaged_async_result<Function, Allocator,
            typename result_of<Function(Args...)>::type>(h)
    {
    }
};

#else // defined(NET_TS_HAS_VARIADIC_TEMPLATES)

template <typename Allocator, typename Result>
class async_result<use_future_t<Allocator>, Result()>
    : public detail::promise_async_result<void(), Allocator>
{
public:
    explicit async_result(
        typename detail::promise_async_result<
            void(), Allocator>::completion_handler_type& h)
        : detail::promise_async_result<void(), Allocator>(h)
    {
    }
};

#endif

```

```

    }
};

template <typename Function, typename Allocator, typename Result>
class async_result<detail::packaged_token<Function, Allocator>, Result()>
    : public detail::packaged_async_result<Function, Allocator,
        typename result_of<Function()>::type>
{
public:
    explicit async_result(
        typename detail::packaged_async_result<Function, Allocator,
            typename result_of<Function()>::type>::completion_handler_type& h)
        : detail::packaged_async_result<Function, Allocator,
            typename result_of<Function()>::type>(h)
    {
    }
};

#define NET_TS_PRIVATE_ASYNC_RESULT_DEF(n) \
    template <typename Allocator, \
        typename Result, NET_TS_VARIADIC_TPARAMS(n)> \
    class async_result<use_future_t<Allocator>, \
        Result(NET_TS_VARIADIC_TARGS(n))> \
        : public detail::promise_async_result< \
            void(NET_TS_VARIADIC_DECAY(n)), Allocator> \
    { \
    public: \
        explicit async_result( \
            typename detail::promise_async_result< \
                void(NET_TS_VARIADIC_DECAY(n)), \
                Allocator>::completion_handler_type& h) \
            : detail::promise_async_result< \
                void(NET_TS_VARIADIC_DECAY(n)), Allocator>(h) \
        { \
        } \
    }; \
    \
    template <typename Function, typename Allocator, \
        typename Result, NET_TS_VARIADIC_TPARAMS(n)> \
    class async_result<detail::packaged_token<Function, Allocator>, \
        Result(NET_TS_VARIADIC_TARGS(n))> \
        : public detail::packaged_async_result<Function, Allocator, \
            typename result_of<Function(NET_TS_VARIADIC_TARGS(n))>::type> \
    { \
    public: \
        explicit async_result( \
            typename detail::packaged_async_result<Function, Allocator, \
                typename result_of<Function(NET_TS_VARIADIC_TARGS(n))>::type \
                >::completion_handler_type& h) \
            : detail::packaged_async_result<Function, Allocator, \
                typename result_of<Function(NET_TS_VARIADIC_TARGS(n))>::type>(h) \
        { \
        } \
    }; \

```

```

}; \
/**/
NET_TS_VARIADIC_GENERATE(NET_TS_PRIVATE_ASYNC_RESULT_DEF)
#undef NET_TS_PRIVATE_ASYNC_RESULT_DEF

#endif // defined(NET_TS_HAS_VARIADIC_TEMPLATES)

#endif // !defined(GENERATING_DOCUMENTATION)

} // inline namespace v1
} // namespace net
} // namespace experimental
} // namespace std

#include <experimental/_net_ts/detail/pop_options.hpp>

#endif // NET_TS_IMPL_USE_FUTURE_HPP

```

12.3 [P1386R2] audio

[P1386R2] has some interesting callbacks. device events report changes in the devices available. These are not so chatty and usually interact with the Ux. requests to fill and empty buffers. These have very stringent restrictions. They must complete within a certain time. They must not access some features and API's. Using [multi function style] Callbacks here shows that these concepts apply smoothly to vastly different domains.

The examples in this section were modified from https://github.com/stdcpp-audio/libstdaudio/blob/master/include/audio_backend/___coreaudio_backend.h

The `stop_token` is used to support unregistering a callback.

`bad_any_callback` would be replaced by a `std::any_callback` for the type-erasure.

NOTE: For the purpose of comparison this paper will use the Callback naming specified in [P1660R0] as an example of [multiple function style](#). The names chosen for a particular expression of the [multiple function style](#) do not affect this proposal.

12.3.1 events

In the [P1660R0] model, registering for the device changed events would look something like:

```

int main () {
    reactor mainThread;

    stop_source stop;

    after(mainThread, 1m) |
        submit([=]() {
            stop.request_stop();
        });

    device_list_changed() |
        transform([]() {
            return list_devices() |
                as<vector>() |

```

```

        via(mainThread) |
        selectDeviceUx();
    }) |
    concat() |
    take_until(stop.get_token()) |
    submit();

    mainThread.run_until(stop.get_token());
}

```

The code below shows how this can be achieved.

```

struct bad_any_callback {
    stop_token stop_;
    function<void()> v_;
    function<void(exception_ptr)> e_;
    function<void()> d_;

    void operator()() {v_();}
    template<class E>
    void error(E e) {e_(make_exception_ptr(e));}
    void error(exception_ptr e) {e_(e);}
    void done() {d_();}
    stop_token get_stop_token() { return stop_; }
};

struct __coreaudio_device_config_listener {
    struct subscription {
        bad_any_callback cb_;
        stop_token_callback sc_;
    };

    static void register_callback(audio_device_list_event event, bad_any_callback cb) {
        static __coreaudio_device_config_listener dcl;
        const auto selector = get_coreaudio_selector(event);
        auto stop = cb.get_stop_token();
        dcl.callbacks[selector] = subscription{
            move(cb),
            stop_token_callback{
                stop,
                [=, &dcl, selector, stop]() {
                    subscription sub = exchange(dcl.callbacks[selector], subscription{});
                    sub.cb_.done();
                }
            }
        };
    }

private:
    map<AudioObjectPropertySelector, subscription> callbacks;

    __coreaudio_device_config_listener() {
        coreaudio_add_internal_callback<kAudioHardwarePropertyDevices>();
        coreaudio_add_internal_callback<kAudioHardwarePropertyDefaultInputDevice>();
        coreaudio_add_internal_callback<kAudioHardwarePropertyDefaultOutputDevice>();
    }
}

```

```

template <AudioObjectPropertySelector selector>
void coreaudio_add_internal_callback() {
    AudioObjectPropertyAddress pa = {
        selector,
        kAudioObjectPropertyScopeGlobal,
        kAudioObjectPropertyElementMaster
    };

    if (!__coreaudio_util::check_error(AudioObjectAddPropertyListener(
        kAudioObjectSystemObject, &pa, &coreaudio_internal_callback<selector>, this))) {
        assert(false); // failed to register device config listener!
    }
}

template <AudioObjectPropertySelector selector>
static OSStatus coreaudio_internal_callback(AudioObjectID device_id,
                                           UInt32 /* inNumberAddresses */,
                                           const AudioObjectPropertyAddress* /* inAddresses */,
                                           void* void_ptr_to_this_listener) {
    __coreaudio_device_config_listener& this_listener = *reinterpret_cast<__coreaudio_device_config_listener*>(void_ptr_to_this_listener);
    this_listener.call<selector>();
    return {};
}

template <AudioObjectPropertySelector selector>
void call() {
    if (auto cb_iter = callbacks.find(selector); cb_iter != callbacks.end()) {
        invoke(cb_iter->second.cb_);
    }
}

static constexpr AudioObjectPropertySelector get_coreaudio_selector(audio_device_list_event event) noexcept {
    switch (event) {
        case audio_device_list_event::device_list_changed:
            return kAudioHardwarePropertyDevices;
        case audio_device_list_event::default_input_device_changed:
            return kAudioHardwarePropertyDefaultInputDevice;
        case audio_device_list_event::default_output_device_changed:
            return kAudioHardwarePropertyDefaultOutputDevice;
        default:
            assert(false); // invalid event!
            return {};
    }
}

template <audio_device_list_event event>
struct __coreaudio_register_callback_sender {
    void submit(bad_any_callback c){
        __coreaudio_device_config_listener::register_callback(event, c);
    }
};

```

```

template <audio_device_list_event event>
struct __coreaudio_register_callback {
    __coreaudio_register_callback_sender<event> operator()() const { return {}; }
};

constexpr inline __coreaudio_register_callback<audio_device_list_event::device_list_changed> device_list_changed{};
constexpr inline __coreaudio_register_callback<audio_device_list_event::default_input_device_changed> default_input_device_changed{};
constexpr inline __coreaudio_register_callback<audio_device_list_event::default_output_device_changed> default_output_device_changed{};

```

12.3.2 buffers

In the [P1660R0] model, handling buffers might look like:

```

stop_source stop;
audio_device d;

d.start() |
transform([](audio_device::starting_audio_device& sd){
    return sd.connect() |
        tap([](
            audio_device&,
            audio_device_io<__coreaudio_native_sample_type>&){
                //..
            });
}) |
concat() |
take_until(stop.get_token()) |
submit();

```

The code below shows how this can be achieved.

```

template<class... Vn>
struct bad_any_callback {
    stop_token stop_;
    function<void(Vn...)> v_;
    function<void(exception_ptr)> e_;
    function<void()> d_;

    void operator()(Vn... vn) {v_(vn...);}
    template<class E>
    void error(E e) {e_(make_exception_ptr(e));}
    void error(exception_ptr e) {e_(e);}
    void done() {d_();}
    stop_token get_stop_token() { return stop_; }

    explicit operator bool() { return v_ && e_ && d_; }
};

struct audio_device {
    template<class... Cb>
    static OSSStatus report_error(OSSStatus st, Cb&... cb) {
        if (st != noErr) {
            (void)((cb.error(st), true) && ... && true);
        }
    }
};

```

```

    return st;
}

using __coreaudio_callback_t = bad_any_callback<
    audio_device&,
    audio_device_io<__coreaudio_native_sample_type>&&>;

struct connect_audio_device {
    audio_device& that_;
    void submit(__coreaudio_callback_t cb) {
        that_.user_callback = cb;
    }
};

struct starting_audio_device {
    audio_device& that_;
    connect_audio_device connect() { return {that_}; }
};

struct start_audio_device {
    audio_device& that_;
    void submit(bad_any_callback<starting_audio_device&> cb) {
        if (!that_.running) {
            auto stop = cb.get_stop_token();

            that_.sc = stop_token_callback{stop, [p=addressof(that_)](){
                if (p->running) {
                    report_error(AudioDeviceStop(
                        p->device_id, _device_callback), p->user_callback);

                    report_error(AudioDeviceDestroyIOProcID(
                        p->device_id, p->proc_id), p->user_callback);

                    p->proc_id = {};
                    p->user_callback = __coreaudio_callback_t{};
                    p->running = false;
                }
            }};

            starting_audio_device starting{that_};
            invoke(cb, starting);

            if (!that_.user_callback) {
                cb.error(audio_device_exception("connect() must be called during device start"));
            }

            if (!__coreaudio_util::check_error(report_error(AudioDeviceCreateIOProcID(
                that_.device_id, _device_callback, this, &that_.proc_id), that_.user_callback, cb)))
                return;

            if (!__coreaudio_util::check_error(report_error(AudioDeviceStart(
                that_.device_id, _device_callback), that_.user_callback, cb))) {
                __coreaudio_util::check_error(AudioDeviceDestroyIOProcID(
                    that_.device_id, that_.proc_id));
            }
        }
    }
};

```

```

        that._proc_id = {};
        return;
    }

    that._running = true;
} else {
    cb.done();
}
}
};

start_audio_device start() {
    return {*this};
}

//...
};

```

13 References

- [N4045] Christopher Kohlhoff. 2014. Library Foundations for Asynchronous Operations, Revision 2.
<https://wg21.link/n4045>
- [N4771] Jonathan Wakely. 2018. Working Draft, C++ Extensions for Networking.
<https://wg21.link/n4771>
- [P1055R0] Kirk Shoop, Eric Niebler, Lee Howes. 2018. A Modest Executor Proposal.
<https://wg21.link/p1055r0>
- [P1341R0] Lewis Baker. 2018. Unifying Asynchronous APIs in the Standard Library.
<https://wg21.link/p1341r0>
- [P1371R0] Sergei Murzin, Michael Park, David Sankel, Dan Sarginson. 2019. Pattern Matching.
<https://wg21.link/p1371r0>
- [P1386R2] Guy Somberg, Guy Davidson, Timur Doumler. 2019. A Standard Audio API for C++: Motivation, Scope, and Basic Design.
<https://wg21.link/p1386r2>
- [P1660R0] Jared Hoberock, Michael Garland, Bryce Adelstein Lelbach, Michał Dominiak, Eric Niebler, Kirk Shoop, Lewis Baker, Lee Howes, David S. Hollman, Gordon Brown. 2019. A Compromise Executor Design Sketch.
<https://wg21.link/p1660r0>
- [P1677R0] Kirk Shoop. 2019. Cancellation is not an Error.
<https://wg21.link/p1677r0>